

UNIT - IV

Model Validation in Classification : Cross Validation - Holdout Method, K-Fold, Stratified K-Fold, Leave-One-Out Cross Validation. Bias-Variance tradeoff, Regularization , Overfitting, Underfitting. **Ensemble Methods:** Boosting, Bagging, Random Forest.



Train vs. Validation vs. Test set

For training and testing purposes of our model, we should have our data broken down into three distinct dataset splits.

The Training Set

It is the set of data that is used to train and make the model learn the hidden features/patterns in the data.

The Validation Set

The validation set is a set of data, separate from the training set, that is used to validate our model performance during training.

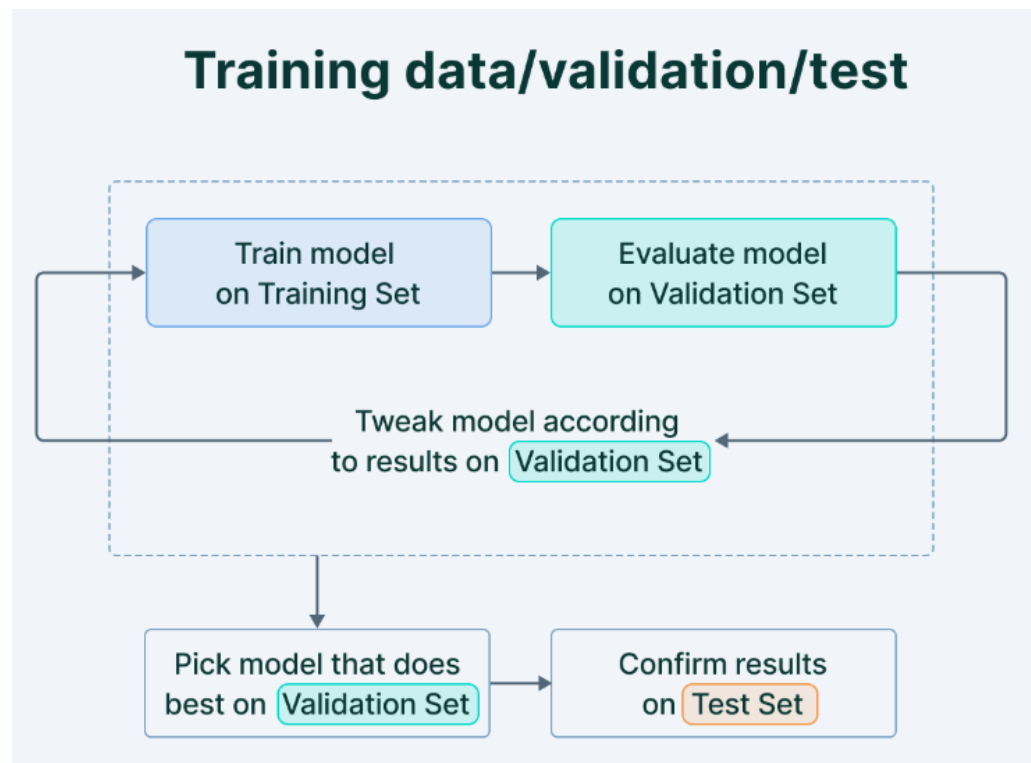
It is like a critic telling us whether [the training](#) is moving in the right direction or not.

The model is trained on the training set, and, simultaneously, the model evaluation is performed on the validation set after every epoch.

The main idea of splitting the dataset into a validation set is to prevent our model from overfitting i.e., the model becomes really good at classifying the samples in the training set but cannot generalize and make accurate classifications on the data it has not seen before.

The Test Set

The test set is a separate set of data used to test the model *after* completing the training.



cross-validation:

The basic purpose of cross-validation is to assess how the model will perform with an unknown data set.

For instance, you are trying to score a goal in an empty goal. It looks pretty easy, and you could even score from a considerable distance too. But the real test starts when there is a goalkeeper and a bunch of defenders. That's why you need to get trained in a real match facing all the heat and still score the goal.

Similarly, a statistical model is trained in such a way that it excels in its efficiency with other unknown data sets using cross-validation.

1.Holdout Method

Holdout Method is the simplest sort of method to evaluate a classifier. In this method, the data set (a collection of data items or examples) is separated into two sets, called the **Training set and Test set**.

A classifier performs function of assigning data items in a given collection to a target category or class.

Example –

E-mails in our inbox being classified into spam and non-spam.

Classifier should be evaluated to find out, its accuracy, error rate, and error estimates. It can be done using various methods. One of most primitive methods in evaluation of classifier is '**Holdout Method**'.

In the holdout method, data set is partitioned, such that – maximum data belongs to training set and remaining data belongs to test set.

Example –

If there are 20 data items present, 12 are placed in training set and remaining 8 are placed in test set.

- After partitioning data set into two sets, training set is used to build a model/classifier.
- After construction of classifier, we use data items in test set, to test accuracy, error rate and error estimate of model/classifier.

Problem :

During partitioning of whole data set into 2 parts i.e., training set and test set, if all data items belonging to class are placed in test set entirely, such that none of data items of class are in training set. It is evident, that model/classifier built, is not trained using data items of class.

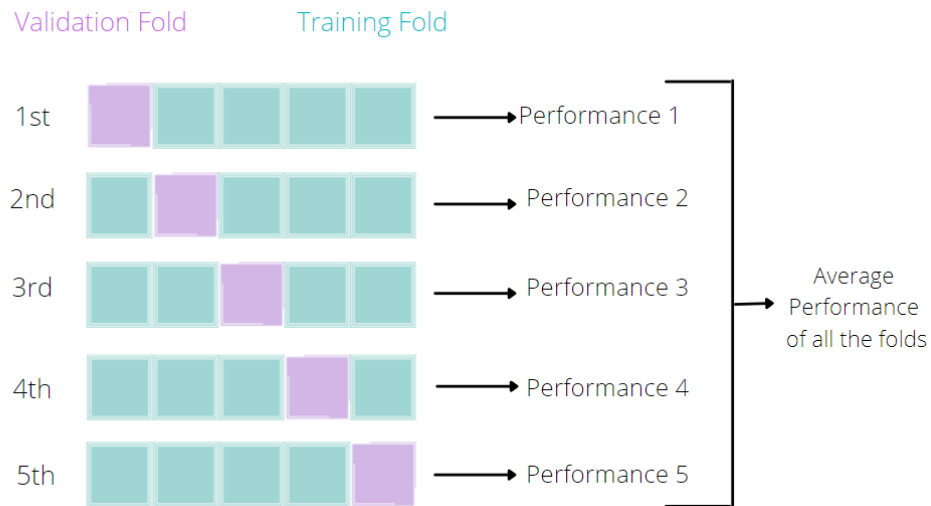
Hold-out cross-validation is the simplest and most common technique. You might not know that it is a **hold-out** method but you certainly use it every day.

The algorithm of hold-out technique:

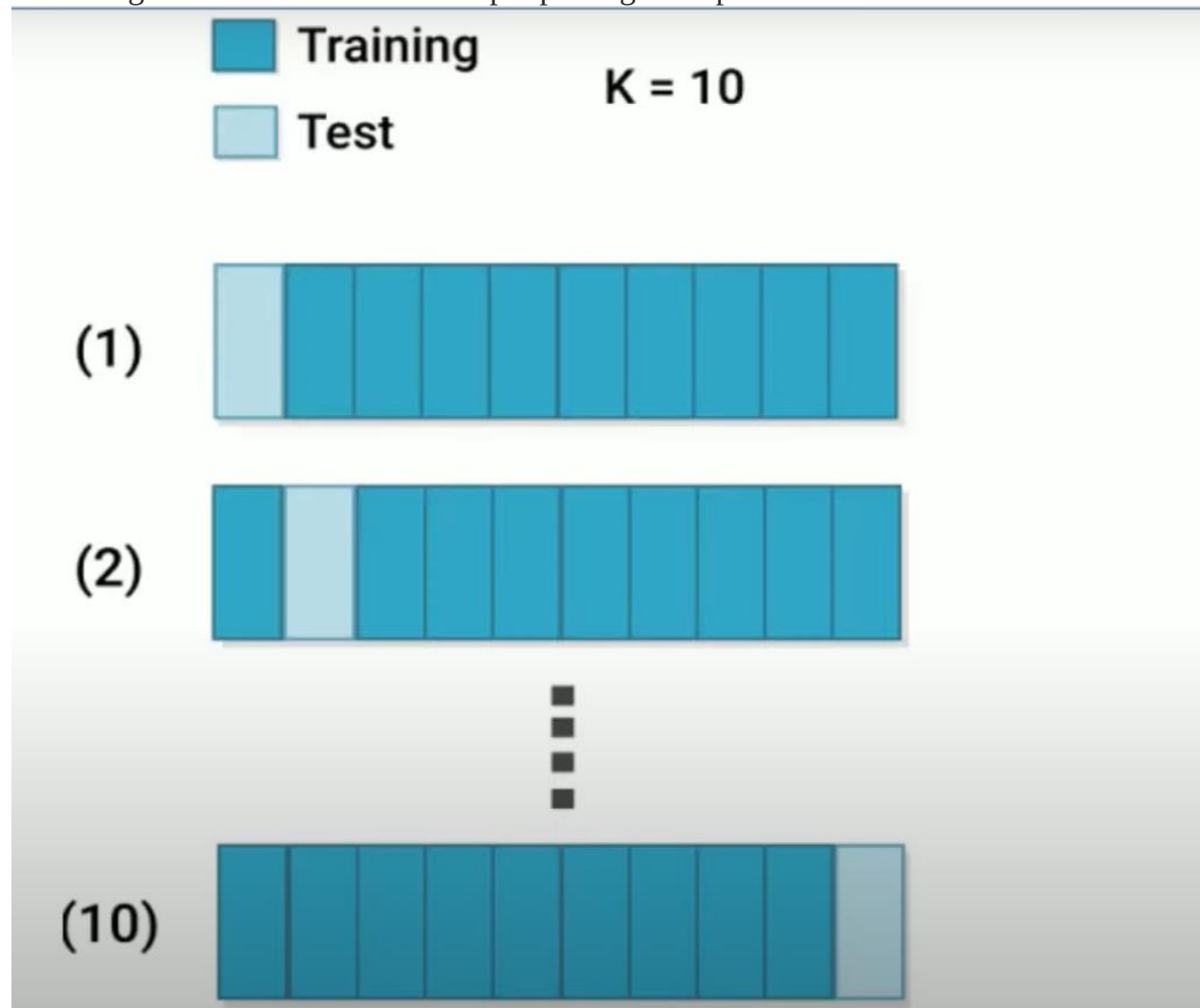
1. Divide the dataset into two parts: the training set and the test set. Usually, 80% of the dataset goes to the training set and 20% to the test set but you may choose any splitting that suits you better
2. Train the model on the training set
3. Validate on the test set
4. Save the result of the validation.



2.K-fold Cross Validation



in K Fold Cross Validation, we split the dataset into k folds, k-1 to train the model and the remaining one to evaluate it. We keep repeating this operation k times.

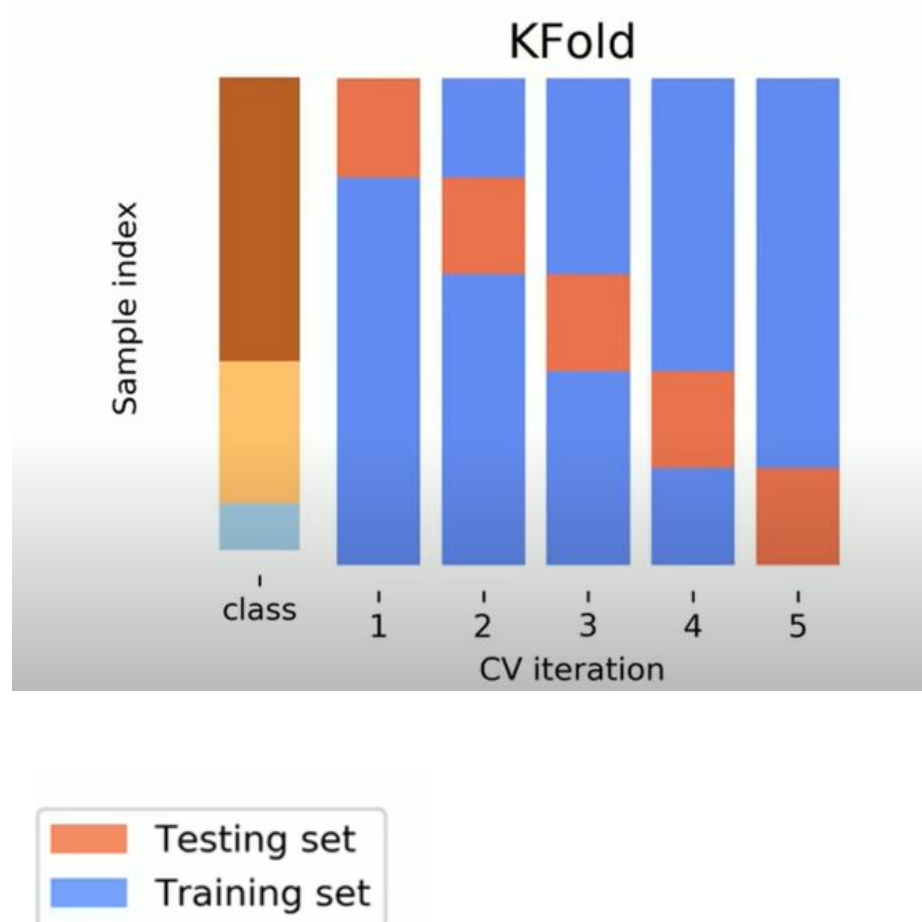


Divide entire data set into k folds(here k folds).

In every iteration 1 fold for testing remaining 9 folds for training.

For all 10 iterations same procedure follows.

Consider average of each iteration performance is the model performance.



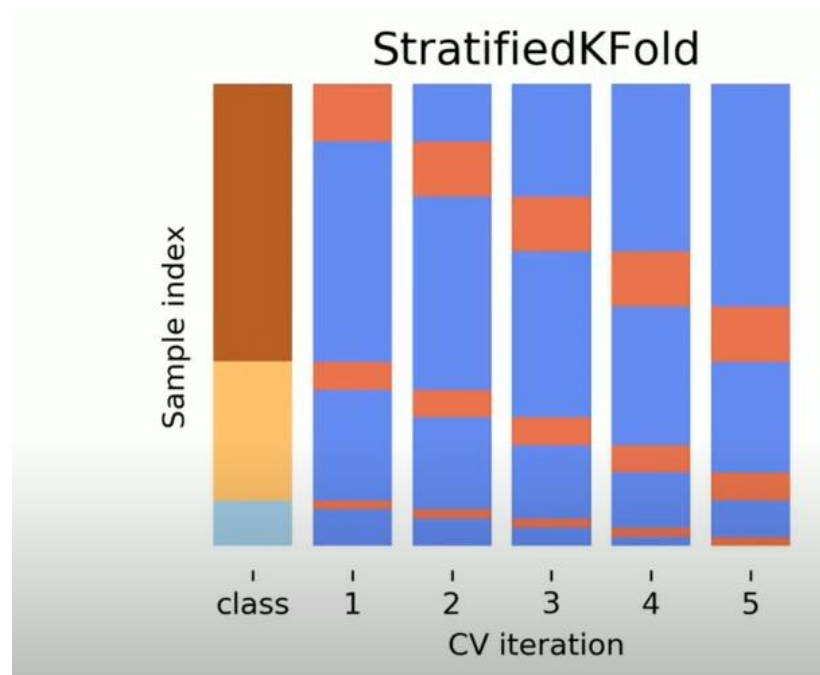
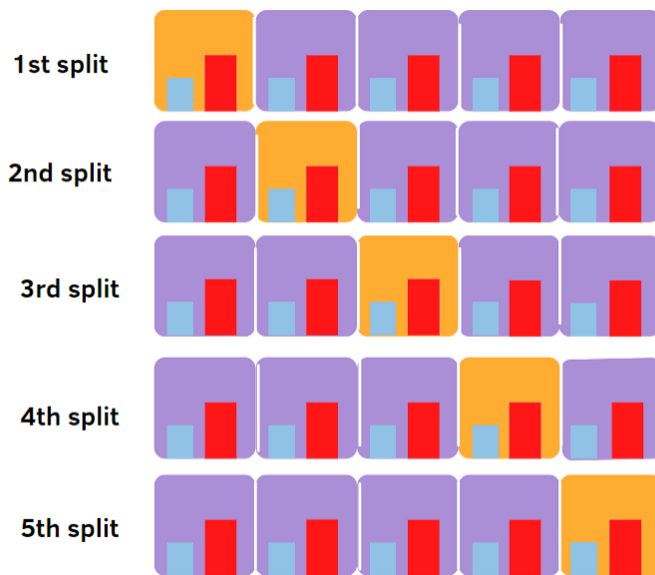
Here we are taking equal representation of the 3 classes.

Using K Fold on a classification problem can be tricky. Since we are randomly shuffling the data and then dividing it into folds, chances are we may get highly imbalanced folds which may cause our training to be biased. For example, let us somehow get a fold that has majority belonging to one class(say positive) and only a few as negative class. This will certainly ruin our training and to avoid this we make stratified folds using stratification.

- 1)Testfold is not repeated
- 2)no overlapping of testfold
- 3)each fold is in testfold and training fold

3.Stratified K Fold Cross Validation

Stratification is the process of rearranging the data so as to ensure that each fold is a good representative of the whole. For example, in a binary classification problem where each class comprises of 50% of the data, it is best to arrange the data such that in every fold, each class comprises of about half the instances.



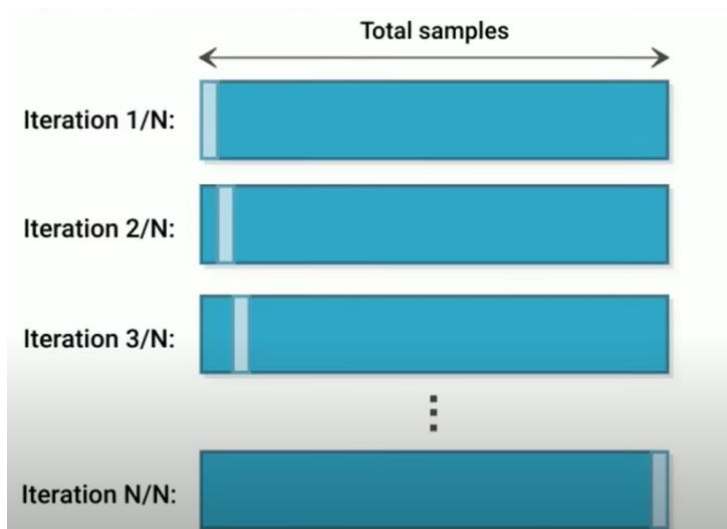


Here equal representation of each class.

4. Leave one out cross validation

Here 1 example for testing, remaining n examples for training.
If we have n examples, n iterations.

Out of 100, 1 example for testing remaining 99 examples for training.
So total 100 iterations.



What is Bias?

To make predictions, our model will analyze our data and find patterns in it. Using these patterns, we can make generalizations about certain instances in our data. Our model after training learns these patterns and applies them to the test set to predict them.

Bias is the difference between our actual and predicted values. Bias is the simple assumptions that our model makes about our data to be able to predict new data.

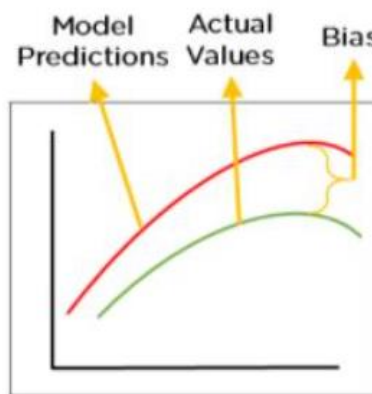
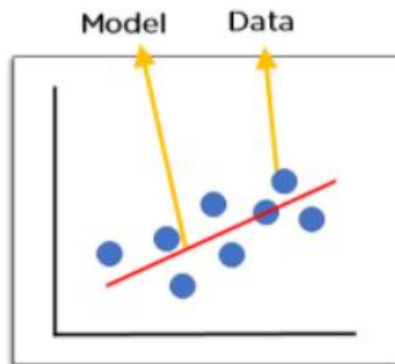


Figure : Bias

When the Bias is high, assumptions made by our model are too basic, the model can't capture the important features of our data. This means that our model hasn't captured patterns in the training data and hence cannot perform well on the testing data too. If this is the case, our model cannot perform on new data and cannot be sent into production.

This instance, where the model cannot find patterns in our training set and hence fails for both seen and unseen data, is called Underfitting.

The below figure shows an example of Underfitting. As we can see, the model has found no patterns in our data and the line of best fit is a straight line that does not pass through any of the data points. The model has failed to train properly on the data given and cannot predict new data either.



Variance:

if our model is allowed to view the data too many times, it will learn very well for only that data. It will capture most patterns in the data, but it will also learn from the unnecessary data present, or from the noise.

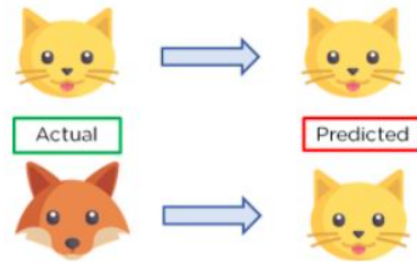


Figure 4: Example of Variance

In the above figure, we can see that our model has learned extremely well for our training data, which has taught it to identify cats. But when given new data, such as the picture of a fox, our model predicts it as a cat, as that is what it has learned. This happens when the Variance is high, our model will capture all the features of the data given to it, including the noise, will tune itself to the data, and predict it very well but when given new data, it cannot predict on it as it is too specific to training data.

Hence, our model will perform really well on testing data and get high accuracy but will fail to perform on new, unseen data. New data may not have the exact same features and the model won't be able to predict it very well. This is called Overfitting.

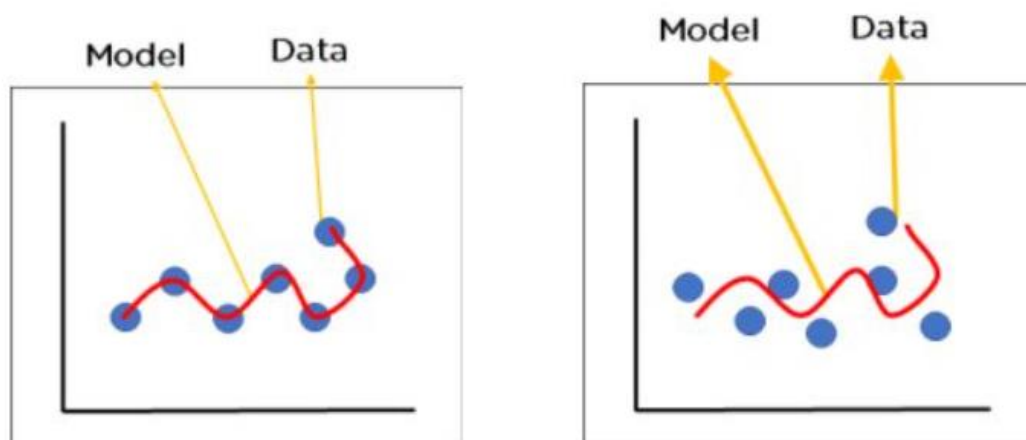
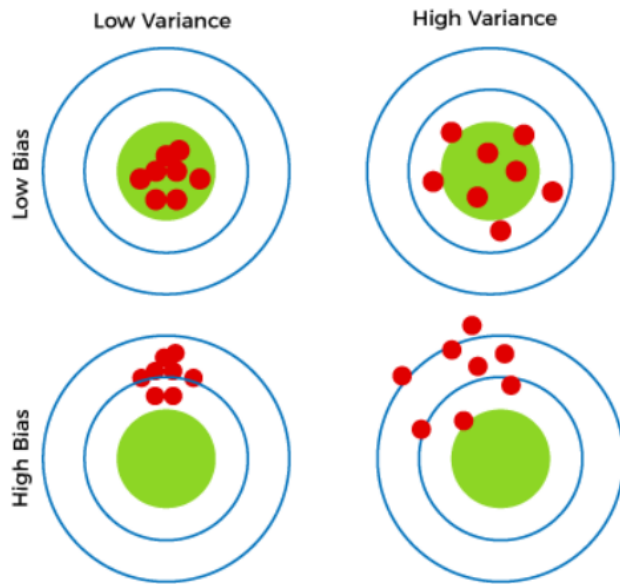


Figure 5: Over-fitted model where we see model performance on, a) training data b) new data

Bias-Variance trade-off.

There are four possible combinations of bias and variances, which are represented by the below diagram:



1. **Low-Bias, Low-Variance:**

The combination of low bias and low variance shows an ideal machine learning model. However, it is not possible practically.

2. **Low-Bias, High-Variance:** With low bias and high variance, model predictions are inconsistent and accurate on average. This case occurs when the model learns with a large number of parameters and hence leads to an **overfitting**

3. **High-Bias, Low-Variance:** With High bias and low variance, predictions are consistent but inaccurate on average. This case occurs when a model does not learn well with the training dataset or uses few numbers of the parameter. It leads to **underfitting** problems in the model.

4. **High-Bias, High-Variance:**

With high bias and high variance, predictions are inconsistent and also inaccurate on average.

While building the machine learning model, it is really important to take care of bias and variance in order to avoid overfitting and underfitting in the model. If the model is very simple with fewer parameters, it may have low variance and high bias. Whereas, if the model has a large number of parameters, it will have high variance and low bias. So, it is required to make a balance between bias and variance errors, and this balance between the bias error and variance error is known as **the Bias-Variance trade-off**.

For an accurate prediction of the model, algorithms need a low variance and low bias.

Bias – Our model will not be trained well with the training data. There will be high training error when we train our model with the data.

Variance – If you train your data on training data and obtain a very low error, upon changing the data and then training the same previous model you experience high error, this is variance.

In variance high testing error, low training error.



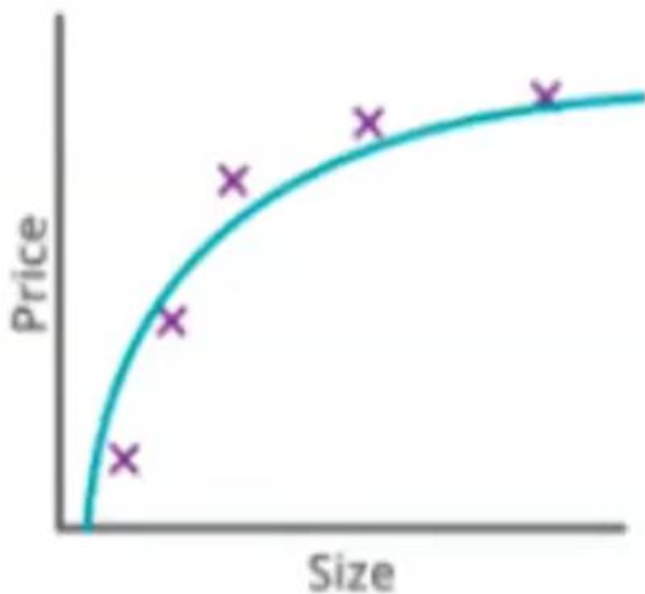
High Training error

Bias - High

High Test error

Variance - High

underfitting



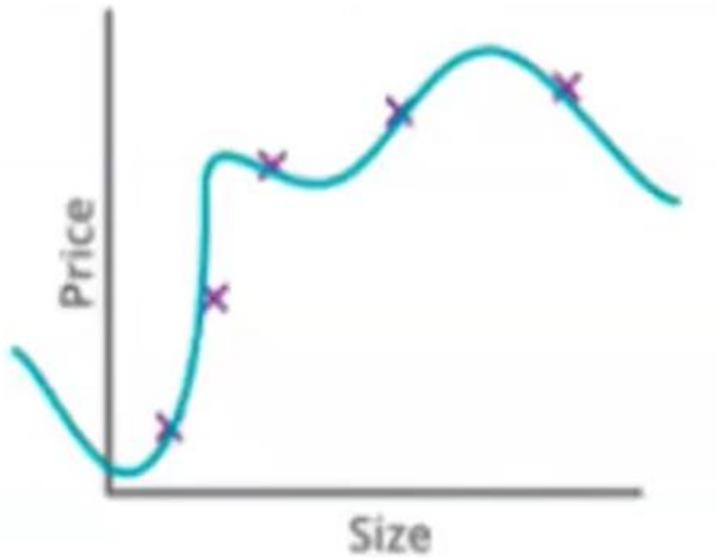
Low Training error

Bias - Low

Low Test error

Variance - Low

Generalized model



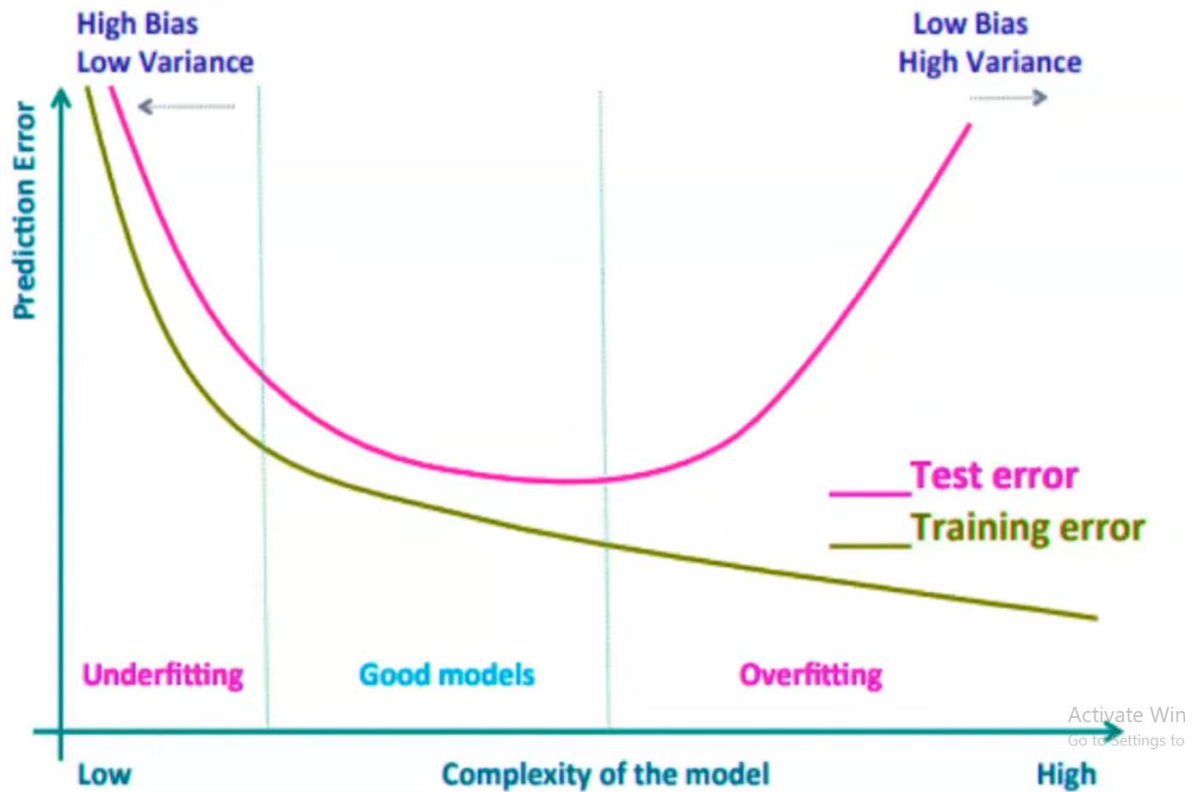
Low Training error

Bias - Low

High Test error

Variance - High

Overfitting



In underfitting, high bias, low variance

In overfitting, low bias, high variance

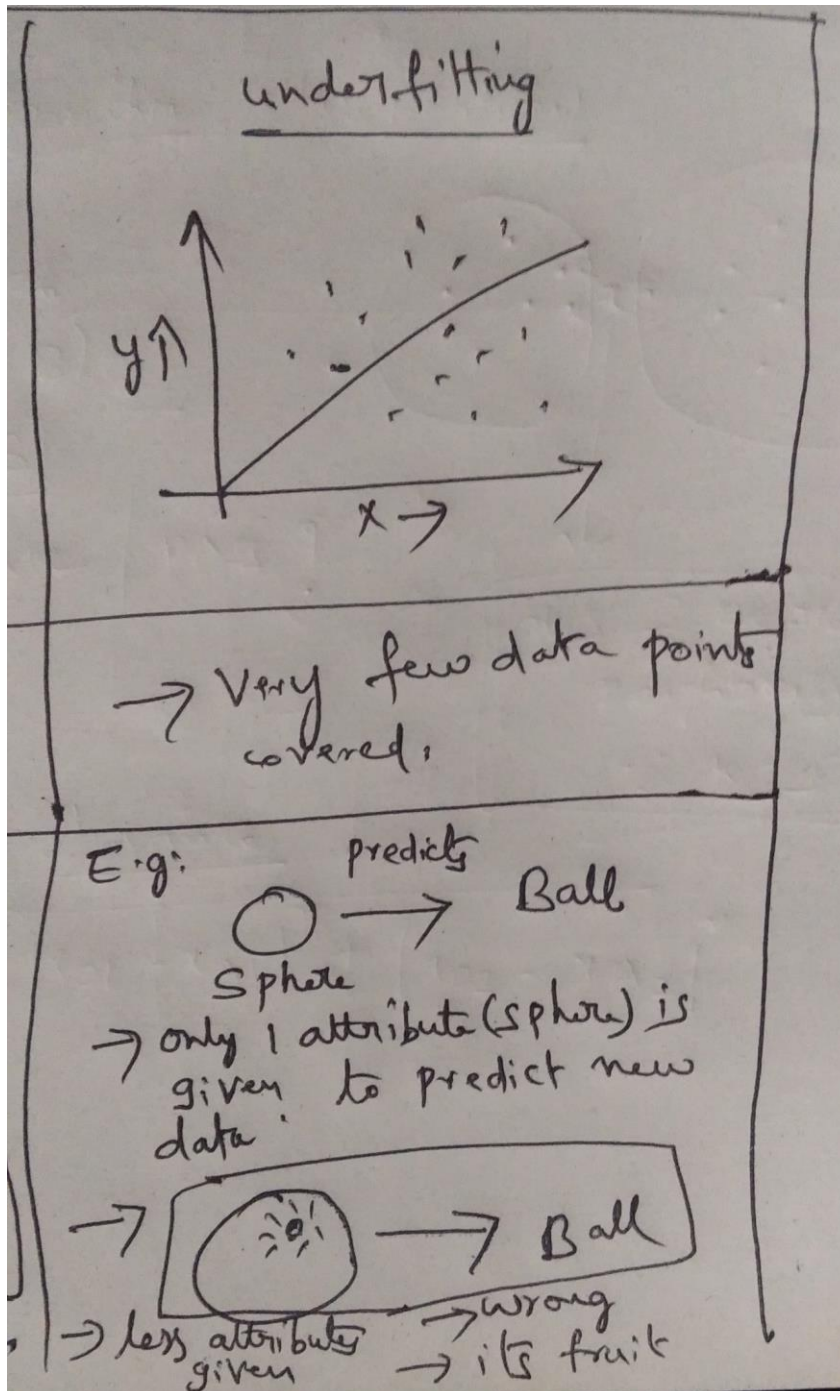
In good model, low bias, low variance

Overfitting:

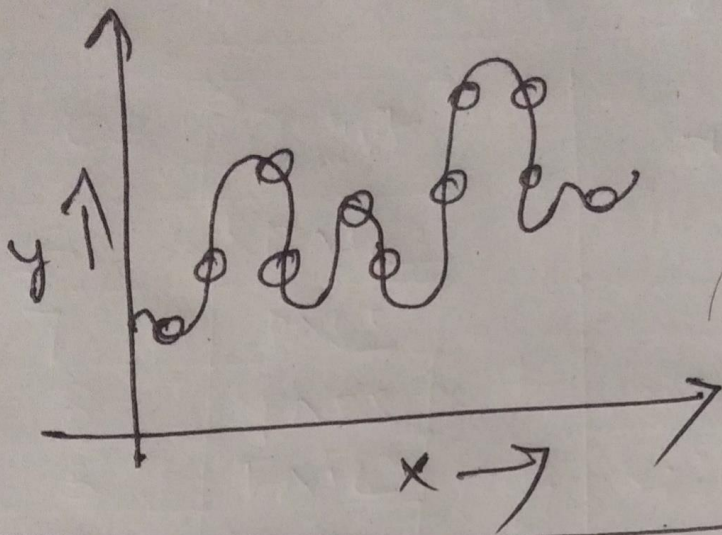
Due to excess knowledge of attributes where model tries to use all attributes (even the least important ones)

Underfitting :

Due to lack of enough attributes, model doesn't have enough knowledge and gives incoherent results



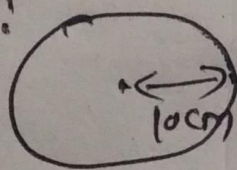
overfitting



→ Every data point is covered.

E.g. attributes $\Rightarrow$ Sphere
play
Can't eat
radius = 5 cm

- more no. of attributes given
- now Training is completed
- Test data!

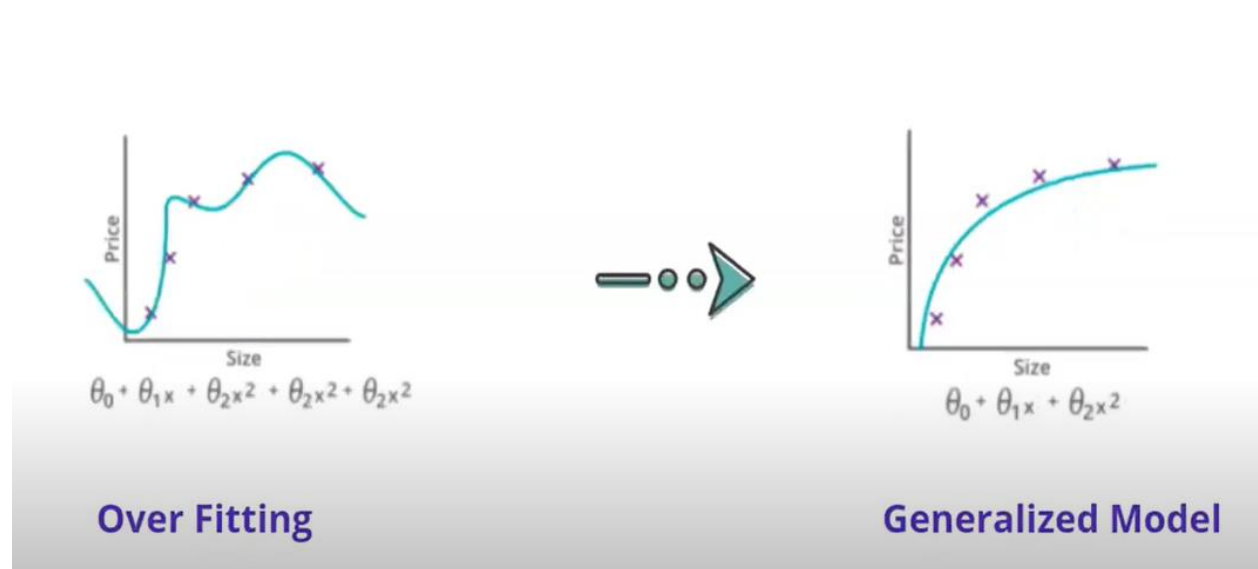


$\Rightarrow$ result: It's
not a
ball
But it's a ball.

Regularization

Regularization converts overfitting model to generalized model.

Consider upto 2 dimension, remaining are nullified to get generalized model.



- Regularization is an approach to address over-fitting in ML.
- Overfitted model fails to generalize estimations on test data
- Regularization reduces the variance of the model

In Lasso regression unimportant features weights are zero.

Lasso regression (short for “Least Absolute Shrinkage and Selection Operator”) is a type of linear regression that is used for feature selection and regularization. Adding a penalty term to the cost function of the linear regression model is a technique used to prevent overfitting. This encourages the model to use fewer variables or features in the final regression equation. The penalty term is based on the absolute values of the coefficients in the equation, which means that some of the coefficients may be set to zero if they are deemed to be less important.

Definition Of Lasso Regression in Machine Learning

Lasso regression is like linear regression, but it uses a technique "**shrinkage**" where the coefficients of determination are shrunk towards **zero**.

Linear regression gives you regression coefficients as observed in the dataset. The lasso regression allows you to shrink or regularize these coefficients to avoid overfitting and make them work better on different datasets.

This type of regression is used when the dataset shows high multicollinearity or when you want to automate variable elimination and **feature selection**.

When To Use Lasso Regression?

Choosing a model depends on the dataset and the problem statement you are dealing with. It is essential to understand the dataset and how features interact with each other.

Lasso regression **penalizes** less important features of your dataset and makes their respective coefficients zero, thereby eliminating them. Thus it provides you with the benefit of feature selection and simple model creation.

So, if the dataset has high dimensionality and high correlation, lasso regression can be used.

$$loss = error(y, \hat{y}) + \lambda \sum_j |\beta_j| \quad \lambda \geq 0, \lambda \propto \text{model bias}, \lambda \propto \frac{1}{\text{model variance}}$$

Here β_j is weights, unimportant features weights become zeroes.

Here we minimize the loss function.

Take correct lambda value.

if lambda is high it leads to underfitting .

if lambda is low it leads to overfitting.

Lasso regression penalty consists of all the estimated parameters. Lambda can be any value between zero to infinity. This value decides how aggressive regularization is performed. It is usually chosen using cross-validation.

Lasso penalizes the sum of absolute values of coefficients. As the lambda value increases, coefficients decrease and eventually **become zero**. This way, lasso regression eliminates insignificant variables from our model.

Our regularized model may have a slightly high bias than linear regression but less variance for future predictions.

L2 Regularization

Overfitting problems may lead to inaccurate and unstable model building. So, a technique that helps minimize the overfitting problem in **machine learning models** is known as regularization.

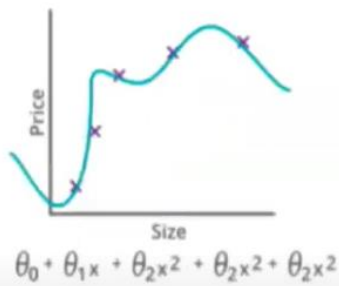
We call it regularization because it keeps the parameters usual or regularized. **Different regression models** use different regularization techniques. The regression model using the **L1** regularization technique is termed as **Lasso regression**.

While the regression model uses **L2** is termed as **Ridge regression**.

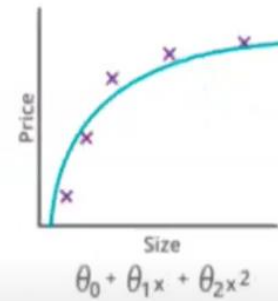
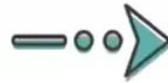
$$loss = error(y, \hat{y}) + \lambda \sum_j \beta_j^2 \quad \lambda \geq 0, \lambda \propto \text{model bias}, \quad \lambda \propto \frac{1}{\text{model variance}}$$

Here we assign very very small value for beta(weights) like 0.02,0.03..

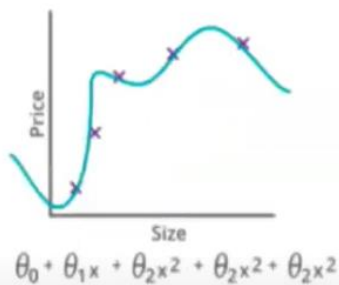
Then no problem, those unimportant features weights are exist but no impact to loss function.



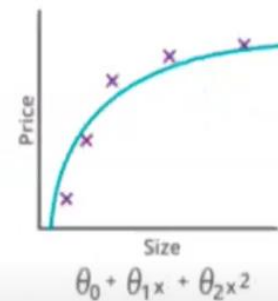
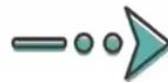
Over Fitting



Generalized Model



Over Fitting



Generalized Model

Ensemble methods

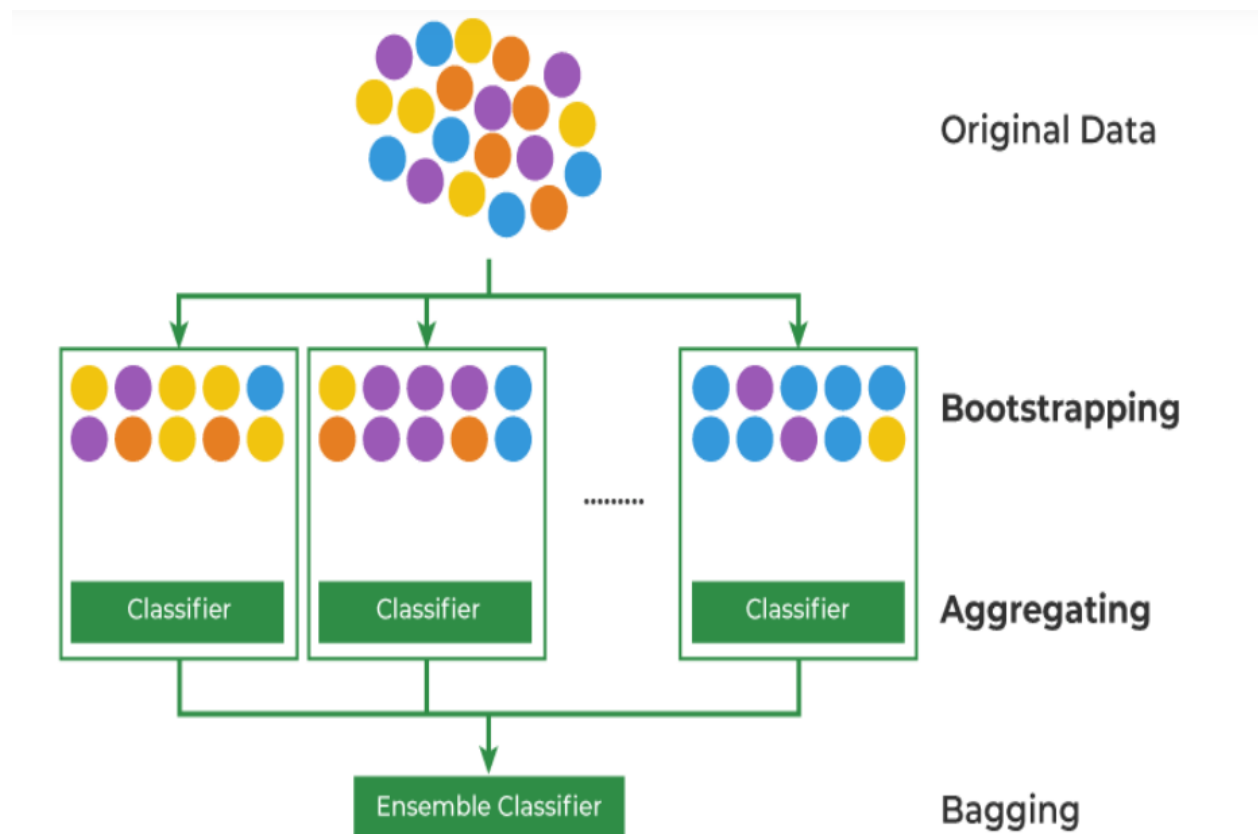
Ensemble methods are techniques that create multiple models and then combine them to produce improved results. Ensemble methods in machine learning usually produce more accurate solutions than a single model would.

1. Bagging

2. Random forest

3. Boosting

1. Bagging



Bootstrap aggregating also called bagging.

It decreases the variance and helps to avoid overfitting.

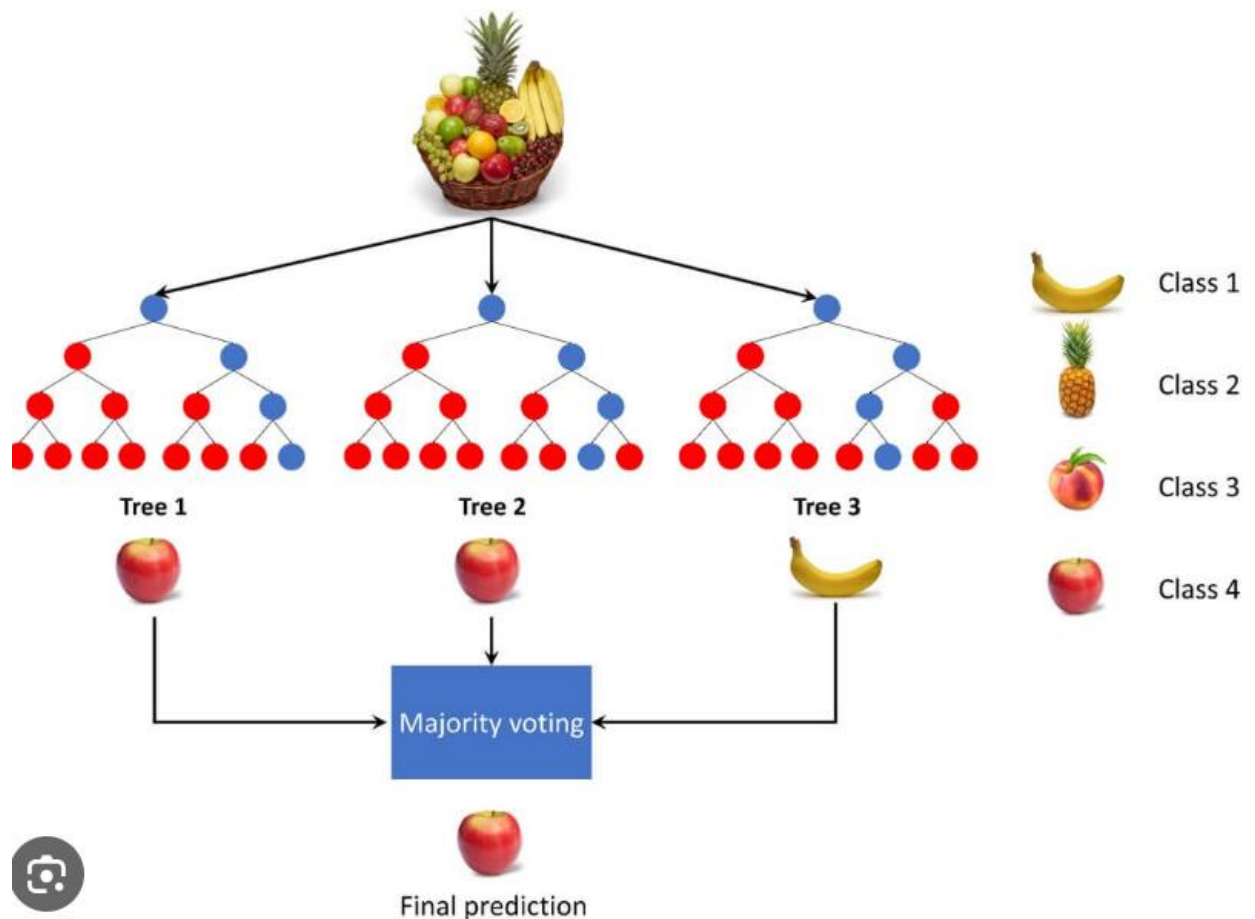
It is usually applied to decision tree methods

Here input data is divided into multiple groups.(n groups).

Each group is given to the same model(KNN,svm,decision tree.....)

Each model gives prediction.predictions will be combined and finally will get the result.

2.Random forest



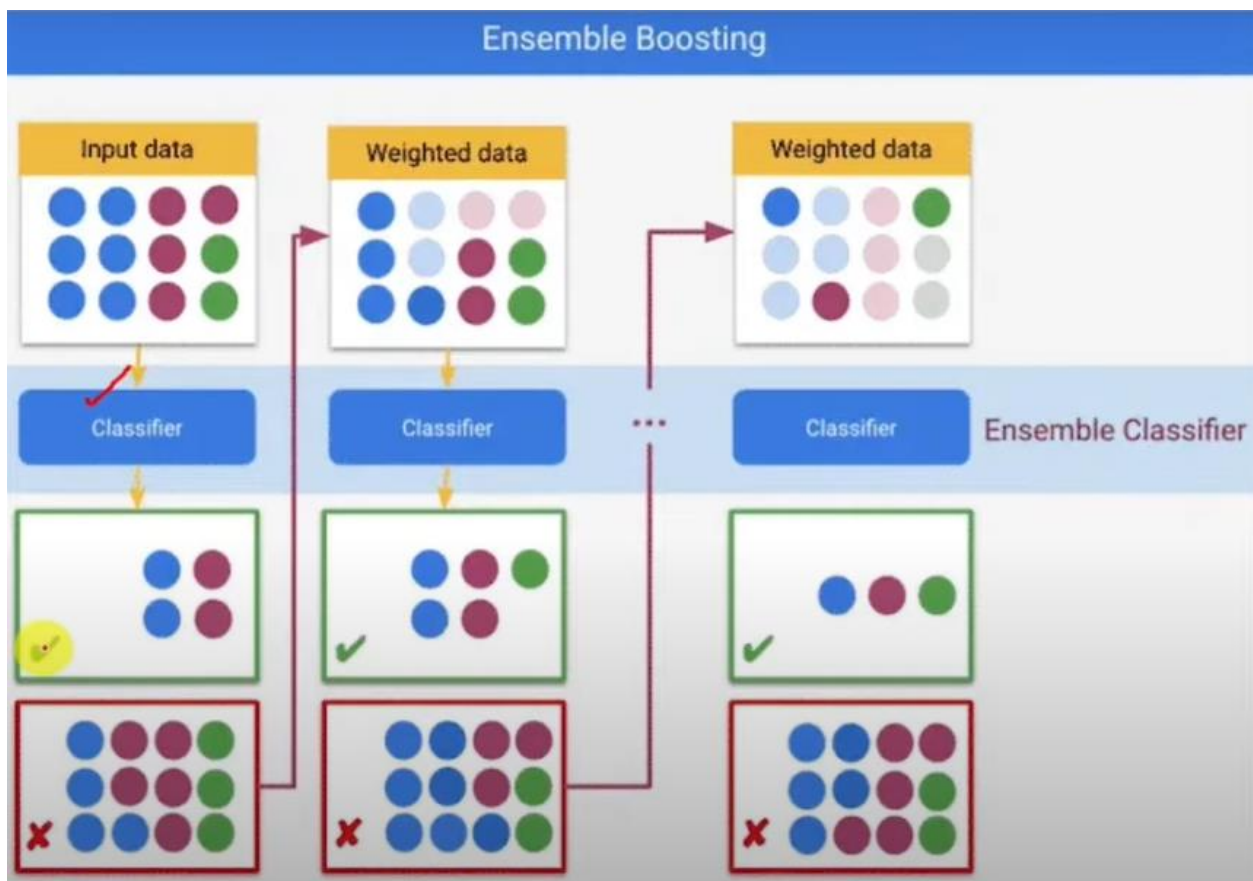
Extension of decision tree algorithm

In decision tree we draw single tree for the given data set based on the particular tree we make the classification of the given data set

In random forest divide input data into multiple groups. each input is given into one decision tree, we will get multiple decision trees.

All decision trees give prediction, average all predictions finally will get final result.

3. Boosting



Boosting builds a strong classifier from the number of weak classifiers.

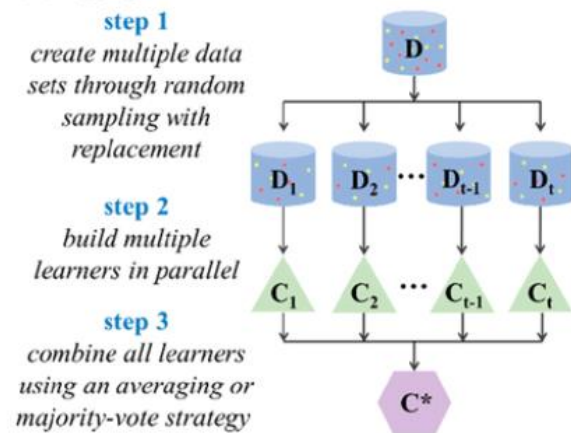
It is done by building a model by using weak models in series

Firstly a model is built from training data(some part is correctly classified ,some part is incorrectly classified)

Consider incorrectly classified data give more weightage to it,try to correct those errors in the second model.

This process is repeated until entire data is correctly classified.

(A) bagging



(B) boosting

